

The fitting procedure of the neural network chain ladder: training, validation and test data

Explainer for `nncl_fit()` in `R/nn_chain_ladder.R` and the scripts `NN_chain_ladder_SynthETIC_fit.R` and `NN_chain_ladder_SynthETIC_analysis.R`

Prepared for Francis

3 October 2026

Contents

Notation used in this document	3
1 What this document covers	4
2 The data and its three parts	4
3 One regression problem per development period	4
4 The split into training and validation rows	5
4.1 Definition	5
4.2 What the cut selects	5
5 The two loss functions	6
6 The fitting procedure	7
6.1 Epochs and gradient steps	7
6.2 Variant A: the paper's training (Listing 2)	7
6.3 Variant B: early stopping (S2)	7
6.4 Variant C: the chain-ladder start (S3, training <code>cl_start</code>)	8
6.5 Variant D: the balance correction (S4)	8
6.6 The procedure as one map	9
7 What each part of the data is used for	9
8 From the fitted factors to the reserve	9
9 The test set and the bias	10

10 One stage and two stages	10
10.1 The two designs	10
10.2 What is gained and lost	11
10.3 What a refit would look like here	12
11 Properties of this particular split	12
12 Where each step is in the code	12
13 Numerical check	13

Notation used in this document

The symbols are those of the companion explainer *The mathematics behind the neural network chain ladder and its chain-ladder start* (cited below as “the companion”); the ones in the lower block are new here.

Symbol	Meaning	Section
I, J	number of accident years, last development year; $I = 20$, $J = I - 1 = 19$	2
i, j	accident year $1, \dots, I$; development year $0, \dots, J$; also the index $1, \dots, J$ of a network	2, 3
$\mathbf{x}$	feature value (the five covariates); as network input a vector in $\mathbb{R}^d$, $d = 14$	2
$C_{i,j}(\mathbf{x})$	cumulative payments up to development year j of the cell $(i, \mathbf{x})$	2
$f_{j-1}(\mathbf{x})$	chain-ladder (CL) factor from development year $j - 1$ to j	3
θ	all parameters of one network; $f(\mathbf{x}; \theta)$ its factor	3
$\mathcal{L}_j, n_j$	learning cells of network j ; their number	3
$c_r, d_r, \mathbf{x}_r$	row r of $\mathcal{L}_j$: $C_{i,j-1}(\mathbf{x})$, $C_{i,j}(\mathbf{x})$ and the feature	3
$\mathcal{T}_j, \mathcal{V}_j$	training rows and validation rows of network j	4
$\hat{f}_{j-1}^{\mathcal{T}}$	homogeneous CL factor on the training rows	4, 6
u	unit of the payments in the fit, $u = 10^6$	5
η	learning rate	6
$\hat{R}, R^{\text{true}}$	reserve estimate; true outstanding payments	8, 9
New in this document		
$\mathcal{O}, \mathcal{O}^c$	observed payments (upper triangles of all cells); future payments (test set)	2
$\prec$	the order of the cells: by accident year, then by the covariates	4
ν	validation fraction, $\nu = 0.1$	4
$n_j^{\mathcal{T}}, n_j^{\mathcal{V}}$	number of training and validation rows	4
$\hat{f}_{j-1}^{\mathcal{L}}, \hat{f}_{j-1}^{\mathcal{V}}$	homogeneous CL factor on all learning cells; on the validation rows	4
$\ell^{\mathcal{T}}(\theta), \ell^{\mathcal{V}}(\theta)$	mean loss on the training rows; on the validation rows	5
B	batch size, $B = 10,000$	6
s	seed (fixes the random initial weights)	6
t, θ_t	epoch (here: gradient step) $0, 1, 2, \dots$; parameters after epoch t	6
U	one update of the optimiser	6
$T_{\max}, p$	maximum number of epochs; patience of early stopping	6
t^*, t_{stop}	epoch whose weights are kept; epoch at which training ends	6
$\hat{\theta}_j$	fitted parameters of network j	6, 7
$\mathcal{D}_i$	diagonal cells of accident year i with a positive latest payment	8

Abbreviations: CL, chain ladder; AY, accident year. “The paper” is Wuthrich (2018), *Neural networks applied to chain-ladder reserving*, European Actuarial Journal 8:407-436; “Paper C” is Gabrielli, Richman and Wuthrich (2020), the bCCNN. Equation numbers of the paper are quoted as your code comments quote them.

1 What this document covers

The companion describes the model, the gradients and the optimisers. This document describes the *procedure* around them: which data each network is fitted on, which data decide when the fitting stops, which data are held back to judge the result, and what the fitted network is therefore a function of. It follows the code line by line and writes each step as a formula.

In one paragraph: every cell contributes an observed part and a future part. The future part is the **test set**; it is never used in fitting. From the observed part each of the $J = 19$ networks draws its **learning cells**; Keras splits them, in the order of the rows, into **training rows** (the first 90%, which enter the gradient) and **validation rows** (the last 10%, which are only monitored). The network that is kept is the one trained on the training rows; there is no second fit on all learning cells.

How the statements were checked. The counts, the factors and the identities of Sections 4 to 7 were recomputed in base R from the saved cells and the saved fits (20 seeds of the one-layer network with the chain-ladder start, and 20 seeds of the paper's network). The results are in Section 13.

2 The data and its three parts

A cell is a pair $(i, \mathbf{x})$ of accident year and feature value; $C_{i,j}(\mathbf{x})$ are its cumulative payments. There are 3,342 cells. At the end of year I a payment of development year j of accident year i is known if and only if $i + j \leq I$. This divides all cumulative payments into

$$\mathcal{O} = \{C_{i,j}(\mathbf{x}) : i + j \leq I\}, \quad \mathcal{O}^c = \{C_{i,j}(\mathbf{x}) : i + j > I, j \leq J\}. \quad (1)$$

$\mathcal{O}$ holds 34,932 values and $\mathcal{O}^c$ 31,908. In the code the logical matrix **observed** marks $\mathcal{O}$.

- $\mathcal{O}$ is all the information the reserving actuary has. Everything in Sections 3 to 8 is a function of $\mathcal{O}$ only.
- $\mathcal{O}^c$ is the **test set**. It is known here because the claims are simulated to full development. It enters in Section 9 and nowhere else.

The training and validation rows of Section 4 are both subsets of $\mathcal{O}$: they are two ways of using the observed data, not a third kind of data.

3 One regression problem per development period

Network $j \in \{1, \dots, J\}$ estimates the factor $f_{j-1}(\mathbf{x})$ from development year $j - 1$ to j under the model

$$\mathbb{E}[C_{i,j}(\mathbf{x}) \mid C_{i,j-1}(\mathbf{x})] = f_{j-1}(\mathbf{x}) C_{i,j-1}(\mathbf{x}), \quad \text{Var}(C_{i,j}(\mathbf{x}) \mid C_{i,j-1}(\mathbf{x})) = \sigma_{j-1}^2 C_{i,j-1}(\mathbf{x}). \quad (2)$$

Its data are the **learning cells**

$$\mathcal{L}_j = \{(i, \mathbf{x}) : i \leq I - j, C_{i,j-1}(\mathbf{x}) > 0\}, \quad n_j = |\mathcal{L}_j|. \quad (3)$$

The condition $i \leq I - j$ is $i + j \leq I$: both $C_{i,j-1}(\mathbf{x})$ and $C_{i,j}(\mathbf{x})$ lie in $\mathcal{O}$. The condition $C_{i,j-1}(\mathbf{x}) > 0$ leaves out the cells that have no volume to develop (they are handled by the zero claims features, Section 8). Row r of $\mathcal{L}_j$ is the triple

$$(\mathbf{x}_r, c_r, d_r) = (\mathbf{x}, C_{i,j-1}(\mathbf{x}), C_{i,j}(\mathbf{x})),$$

and the network is the function $f(\mathbf{x}; \theta) = \exp(\beta_0 + \beta^\top \mathbf{z}(\mathbf{x}))$ of the companion, Section 4. The J networks share nothing: each has its own parameters, its own learning cells and its own split.

In the code: `learn_rows[[j]]` is $\mathcal{L}_j$, `cum[r, j]` is c_r and `cum[r, j + 1]` is d_r .

4 The split into training and validation rows

4.1 Definition

The cells are stored in a fixed order $\prec$: by accident year first, and within an accident year by the covariates in the order Legal Representation, Injury Severity, Age of Claimant, Vehicle type, Business use (`keyby = c("i", features)` in the fit script). Number the rows of $\mathcal{L}_j$ in this order, $r = 1, \dots, n_j$.

Keras is called with `validation_split = 0.1`. It does not sample: it cuts the rows at a fixed position,

$$n_j^\mathcal{T} = \lfloor (1 - \nu) n_j \rfloor, \quad \mathcal{T}_j = \{r : r \leq n_j^\mathcal{T}\}, \quad \mathcal{V}_j = \{r : r > n_j^\mathcal{T}\}, \quad \nu = 0.1, \quad (4)$$

so $n_j^\mathcal{V} = n_j - n_j^\mathcal{T}$. (Keras shuffles the training rows between epochs, but only after this cut and only among the training rows.) `nncl_fit()` repeats the cut for its own bookkeeping:

```
train <- seq_len(floor(n * (1 - param$validation_split))) # Keras's split
vali <- setdiff(seq_len(n), train)
```

The split depends on j and on nothing else: it is the same for every seed, optimiser, architecture and training variant.

4.2 What the cut selects

Because of the order $\prec$, the last rows of $\mathcal{L}_j$ are the cells of its latest accident year $I - j$, and within that year the cells that come last in the covariate order. Two properties follow.

A hold-out in time. $\mathcal{V}_j$ consists of the latest accident years that network j can see. If accident year $I - j$ has more than $n_j^\mathcal{V}$ learning cells, then $\mathcal{V}_j$ is a part of that single year; otherwise it is that year plus the end of the year before. With roughly 160 cells per accident year, the first case holds as soon as $0.1 n_j \leq 160$, that is from $j = 10$ on.

A slice of the covariates. When only part of an accident year is cut off, it is the end of the covariate order, not a random part. Legal Representation is the first covariate and has the labels N, Y in this order, so the end of an accident year consists of cells with Legal Representation Y, and among those of the highest Injury Severity. For $j = 1$ the 131 validation rows of accident year 18 all have Legal Representation Y; for $j = 19$ the 16 validation rows all have Legal Representation Y and Injury Severity 6.

j	n_j	$n_j^{\mathcal{T}}$	$n_j^{\mathcal{V}}$	AYs in $\mathcal{L}_j$	AYs in $\mathcal{V}_j$	volume share of $\mathcal{V}_j$	$\hat{f}_{j-1}^{\mathcal{T}}$	$\hat{f}_{j-1}^{\mathcal{V}}$	$\hat{f}_{j-1}^{\mathcal{L}}$
1	2,895	2,605	290	1-19	18-19	12.1%	2.53498	2.47519	2.52775
2	2,952	2,656	296	1-18	17-18	12.3%	1.32669	1.32340	1.32628
3	2,817	2,535	282	1-17	16-17	11.7%	1.13756	1.14169	1.13804
4	2,670	2,403	267	1-16	15-16	10.7%	1.07428	1.07182	1.07402
5	2,502	2,251	251	1-15	14-15	10.6%	1.04151	1.04814	1.04221
6	2,336	2,102	234	1-14	13-14	9.9%	1.02173	1.02858	1.02241
7	2,164	1,947	217	1-13	12-13	9.9%	1.01542	1.01692	1.01557
8	1,997	1,797	200	1-12	11-12	9.6%	1.01009	1.00948	1.01003
9	1,826	1,643	183	1-11	10-11	10.1%	1.00685	1.00279	1.00643
10	1,664	1,497	167	1-10	10	10.9%	1.00503	1.01251	1.00585
11	1,496	1,346	150	1-9	9	11.8%	1.00245	1.00128	1.00231
12	1,329	1,196	133	1-8	8	10.8%	1.00221	1.00245	1.00224
13	1,157	1,041	116	1-7	7	11.4%	1.00131	1.00194	1.00139
14	984	885	99	1-6	6	11.1%	1.00068	1.00096	1.00071
15	821	738	83	1-5	5	9.2%	1.00054	1.00024	1.00051
16	655	589	66	1-4	4	7.9%	1.00070	1.00018	1.00066
17	486	437	49	1-3	3	10.1%	1.00118	1.00008	1.00107
18	322	289	33	1-2	2	3.4%	1.00000	1.00000	1.00000
19	159	143	16	1	1	0.3%	1.00001	1.00000	1.00001

The volume share is $\sum_{\mathcal{V}_j} c_r / \sum_{\mathcal{L}_j} c_r$. The last three columns are the homogeneous factors $\sum d_r / \sum c_r$ over the training rows, the validation rows and all learning cells; they satisfy

$$\hat{f}_{j-1}^{\mathcal{L}} = (1 - \omega_j) \hat{f}_{j-1}^{\mathcal{T}} + \omega_j \hat{f}_{j-1}^{\mathcal{V}}, \quad \omega_j = \frac{\sum_{\mathcal{V}_j} c_r}{\sum_{\mathcal{L}_j} c_r}, \quad (5)$$

the volume-weighted average of the two parts. A validation factor that differs from the training factor (for $j = 1$: 2.475 against 2.535) means that the latest accident years, or the covariate slice that is held out, developed differently from the rest.

5 The two loss functions

For a parameter θ of network j define the mean weighted squared losses

$$\ell^{\mathcal{T}}(\theta) = \frac{1}{n_j^{\mathcal{T}}} \sum_{r \in \mathcal{T}_j} \frac{(d_r - f(\mathbf{x}_r; \theta) c_r)^2}{c_r}, \quad \ell^{\mathcal{V}}(\theta) = \frac{1}{n_j^{\mathcal{V}}} \sum_{r \in \mathcal{V}_j} \frac{(d_r - f(\mathbf{x}_r; \theta) c_r)^2}{c_r}. \quad (6)$$

These are Keras's `loss` and `val_loss`: with the response $y_r = d_r / \sqrt{c_r}$ and the volume $w_r = \sqrt{c_r}$ the ordinary mean squared error $(y_r - f w_r)^2$ is the summand of (6) (companion, Section 5). The fit is done with the payments in millions, which divides both losses by $u = 10^6$ and changes nothing else.

The weight $1/c_r$ is the inverse of the variance in (2). The two losses are the same function evaluated on two disjoint sets of rows; they differ in what they are used for.

6 The fitting procedure

One network is fitted by one call of `nncl_fit()`. The steps are the same for all variants; the variants differ in the starting point θ_0 , in the number of epochs and in the rule that picks the final parameters.

6.1 Epochs and gradient steps

An epoch is one pass over the training rows in batches of size B , that is $\lceil n_j^T/B \rceil$ gradient steps. Here $B = 10,000 > n_j^T$ for every j (the largest is 2,656), so

one epoch = one gradient step on all training rows.

Write U for one update of the chosen optimiser (companion, Section 9). The parameters after epoch t are

$$\theta_t = U\left(\theta_{t-1}, \nabla_{\theta} \ell^T(\theta_{t-1}), \text{state}_{t-1}\right), \quad t = 1, 2, \dots \quad (7)$$

Only ℓ^T appears in (7). After each epoch Keras also evaluates $\ell^V(\theta_t)$ and stores it; this evaluation changes no parameter.

6.2 Variant A: the paper's training (Listing 2)

- Start: all weights random (drawn with the seed s), all intercepts 0.
- Epochs: $T_{\max} = 100$, no stopping rule.
- Result: the parameters after the last epoch,

$$\hat{\theta}_j = \theta_{100}. \quad (8)$$

The validation losses $\ell^V(\theta_1), \dots, \ell^V(\theta_{100})$ are recorded (they are the green points of the figure of the losses for $j = 1$) but they decide nothing. In this variant the validation rows are simply 10% of the observed data that are not used.

6.3 Variant B: early stopping (S2)

- Start: as in variant A.
- Epochs: at most $T_{\max} = 500$, with Keras's early-stopping callback: monitor ℓ^V , patience $p = 20$, `restore_best_weights = TRUE`.

After epoch t let

$$t^*(t) = \min \left\{ s \leq t : \ell^V(\theta_s) = \min_{1 \leq s' \leq t} \ell^V(\theta_{s'}) \right\}$$

be the (first) best epoch so far. Training ends at

$$t_{\text{stop}} = \min \{ t : t - t^*(t) \geq p \} \wedge T_{\max}, \quad (9)$$

the first epoch at which the validation loss has not improved for p epochs, or $T_{\max}$ if that never happens. The weights of the best epoch are put back:

$$t^* = t^*(t_{\text{stop}}), \quad \hat{\theta}_j = \theta_{t^*}. \quad (10)$$

In the tables `epochs_used` is t^* and `epochs_run` is t_{stop} ; whenever training stops before $T_{\max}$, $t_{\text{stop}} = t^* + p$.

6.4 Variant C: the chain-ladder start (S3, training `cl_start`)

- Start: hidden layers random (seed s); output neuron set to $\beta = \mathbf{0}$ and $\beta_0 = \log \hat{f}_{j-1}^{\mathcal{T}}$, with

$$\hat{f}_{j-1}^{\mathcal{T}} = \frac{\sum_{r \in \mathcal{T}_j} d_r}{\sum_{r \in \mathcal{T}_j} c_r}. \quad (11)$$

So $f(\mathbf{x}; \theta_0) = \hat{f}_{j-1}^{\mathcal{T}}$ for all $\mathbf{x}$: the network starts as the homogeneous chain ladder of its training rows (companion, Section 7). - Epochs and stopping: as in variant B, giving t^* by (9)-(10). - One more comparison. Keras compares the epochs $t \geq 1$ among themselves and never looks at the start. `mnc1_fit()` evaluates $\ell^{\mathcal{V}}(\theta_0)$ before training (`mu_start`) and keeps the start if nothing beat it:

$$\hat{\theta}_j = \begin{cases} \theta_0 & \text{if } \min_{1 \leq t \leq t_{\text{stop}}} \ell^{\mathcal{V}}(\theta_t) \geq \ell^{\mathcal{V}}(\theta_0), \\ \theta_{t^*} & \text{otherwise.} \end{cases} \quad (12)$$

In the first case `epochs_used` is set to 0.

Rule (12) gives the guarantee

$$\ell^{\mathcal{V}}(\hat{\theta}_j) \leq \ell^{\mathcal{V}}(\theta_0) = \frac{1}{n_j^{\mathcal{V}}} \sum_{r \in \mathcal{V}_j} \frac{(d_r - \hat{f}_{j-1}^{\mathcal{T}} c_r)^2}{c_r} : \quad (13)$$

on its validation rows the fitted network is never worse than the chain-ladder factor of its training rows.

6.5 Variant D: the balance correction (S4)

S4 takes the fitted network of variant C and multiplies its factors by

$$c_j = \frac{\sum_{r \in \mathcal{T}_j} d_r}{\sum_{r \in \mathcal{T}_j} f(\mathbf{x}_r; \hat{\theta}_j) c_r},$$

again a quantity of the training rows only (companion, Section 10).

6.6 The procedure as one map

All four variants are a map from the training rows, the validation rows and the seed to the fitted parameters. Collecting (7)-(12),

$$\hat{\theta}_j = \theta_{t^*}(\mathcal{T}_j, s) \quad \text{with} \quad t^* = t^*(\mathcal{T}_j, \mathcal{V}_j, s) \in \{0, 1, \dots, T_{\max}\}. \quad (14)$$

The path $\theta_0, \theta_1, \theta_2, \dots$ is determined by the training rows and the seed alone. The validation rows choose one point on that path. In variant A that choice is fixed in advance ($t^* = 100$) and the validation rows do not enter at all.

7 What each part of the data is used for

	enters the gradient	sets the start	chooses t^*	measures the result
training rows $\mathcal{T}_j$	yes	variant C: $\hat{f}_{j-1}^{\mathcal{T}}$	no	in-sample loss
validation rows $\mathcal{V}_j$	no	no	variants B, C	validation loss
test set $\mathcal{O}^c$	no	no	no	bias of the reserve

The reported losses. With the fitted factors $\hat{f}_r = f(\mathbf{x}_r; \hat{\theta}_j)$, `nnc1_fit()` returns the unnormalised sums (in the payments' unit, after the script multiplies by u)

$$L_j = \sum_{r \in \mathcal{L}_j} \frac{(d_r - \hat{f}_r c_r)^2}{c_r} = \underbrace{\sum_{r \in \mathcal{T}_j} \frac{(d_r - \hat{f}_r c_r)^2}{c_r}}_{\text{loss_train} = n_j^T \ell^{\mathcal{T}}(\hat{\theta}_j)} + \underbrace{\sum_{r \in \mathcal{V}_j} \frac{(d_r - \hat{f}_r c_r)^2}{c_r}}_{\text{loss_vali} = n_j^{\mathcal{V}} \ell^{\mathcal{V}}(\hat{\theta}_j)}. \quad (15)$$

`loss` is the paper's L'_j (6.1) over all learning cells, which is why it mixes rows the network was trained on with rows it was not.

The validation loss is optimistic in variants B and C. There $\hat{\theta}_j$ is the point of the path with the *smallest* validation loss, so

$$\ell^{\mathcal{V}}(\hat{\theta}_j) = \min_t \ell^{\mathcal{V}}(\theta_t) \leq \ell^{\mathcal{V}}(\theta_{t'}) \quad \text{for every fixed } t',$$

and the minimum of noisy quantities is, on average, below the expected loss of any single one of them. The validation loss of an early-stopped network is a fair comparison between networks that were all stopped the same way; it is not an unbiased estimate of the loss on new data. In variant A it is a clean out-of-sample loss, because nothing was chosen with it.

8 From the fitted factors to the reserve

The networks are applied to the **diagonal cells**

$$\mathcal{D}_i = \{\mathbf{x} : C_{i, I-i}(\mathbf{x}) > 0\}, \quad i = 2, \dots, I,$$

3,167 cells in total (`diag_rows`). A cell of accident year i has reached development year $I - i$ and needs the networks $I - i + 1, \dots, J$:

$$\hat{R} = \sum_{i=2}^I \sum_{\mathbf{x} \in \mathcal{D}_i} C_{i,I-i}(\mathbf{x}) \left(\prod_{j=I-i+1}^J f(\mathbf{x}; \hat{\theta}_j) - 1 \right) + \hat{R}_2, \quad (16)$$

where $\hat{R}_2$ is the reserve of the 16 diagonal cells with $C_{i,I-i}(\mathbf{x}) = 0$ from the zero claims features (companion, Section 12). $\hat{R}_2$ is computed from `cum_obs`, the matrix of cumulative payments with everything outside $\mathcal{O}$ set to NA.

Two remarks on how (16) relates to the split.

1. The diagonal cells are not a data set of the fitting. A diagonal cell of accident year i is a learning cell of the networks $1, \dots, I - i$ (as a training or a validation row, for the periods in which it has a positive volume), and it is the *input* of the networks $I - i + 1, \dots, J$, which have never seen accident year i at all, since $\mathcal{L}_j$ only contains accident years up to $I - j < i$.
2. Every factor in (16) is therefore an extrapolation in time: network j is trained on accident years up to $I - j$ (less its validation rows) and applied to the accident years after $I - j$. The feature $\mathbf{x}$ is what carries the factor over; the accident year is not an input of the network.

All of $\hat{R}$ is a function of $\mathcal{O}$ and of the seed.

9 The test set and the bias

The true outstanding payments are

$$R^{\text{true}} = \sum_{i=2}^I \sum_{\mathbf{x}} (C_{i,J}(\mathbf{x}) - C_{i,I-i}(\mathbf{x})) = 9,626.8 \text{ million}, \quad (17)$$

the sum over the lower triangles of all cells, the claims reported after year I included (`cells$true` in the analysis script). Payments after development year J (38.5 million) are in no triangle and are left out of both sides.

The **bias** of a run is $\hat{R} - R^{\text{true}}$, reported in percent of R^{true} . Since $\hat{R}$ depends on $\mathcal{O}$ only and R^{true} on $\mathcal{O}^c$, the bias of a single run that was specified in advance is an honest out-of-sample error.

That stops being true as soon as the test set is used to choose. Picking, among the 60 combinations of hidden layers, optimiser and training, the one with the smallest bias would be a selection on $\mathcal{O}^c$, and its bias would be optimistic for the same reason as the validation loss above. The tables report every combination side by side for that reason; a choice between them has to rest on the validation losses, the spread over the seeds, or an argument, not on the bias.

10 One stage and two stages

10.1 The two designs

Your bCCNN script (Paper C) and the neural network chain ladder use the held-out data differently. In the notation of (14), write $\theta_t(\mathcal{A}, s)$ for the parameters after t steps of training on a set of rows $\mathcal{A}$.

One stage (this code, the paper’s Listing 2).

$$t^* = t^*(\mathcal{T}_j, \mathcal{V}_j, s), \quad \hat{\theta}_j = \theta_{t^*}(\mathcal{T}_j, s). \quad (18)$$

The validated network is the final network.

Two stages (bCCNN with final_fit: refit). A first run on a training part $\mathcal{T}$ of the observed data finds the number of steps t^* with the smallest loss on a validation part $\mathcal{V}$. A second run starts again and trains on *all* observed data for t^* steps:

$$t^* = t^*(\mathcal{T}, \mathcal{V}, s), \quad \tilde{\theta} = \theta_{t^*}(\mathcal{T} \cup \mathcal{V}, s). \quad (19)$$

The first run only answers the question “how many steps?”; the second run produces the reserves.

10.2 What is gained and lost

The one-stage network does not use the validation rows for estimation. By (14) they enter through the integer t^* only. For the factors this means that the latest accident years of every development period, about a tenth of the volume, carry no weight in the gradient.

The effect can be made exact for the homogeneous factor. Under (2), with $\sigma^2 = \sigma_{j-1}^2$,

$$\text{Var}(\hat{f}_{j-1}^{\mathcal{T}}) = \frac{\sigma^2}{\sum_{\mathcal{T}_j} c_r}, \quad \text{Var}(\hat{f}_{j-1}^{\mathcal{L}}) = \frac{\sigma^2}{\sum_{\mathcal{L}_j} c_r}, \quad \frac{\text{Var}(\hat{f}_{j-1}^{\mathcal{T}})}{\text{Var}(\hat{f}_{j-1}^{\mathcal{L}})} = \frac{1}{1 - \omega_j}, \quad (20)$$

with the volume share ω_j of (5). On your data this ratio lies between 1.09 and 1.14 for $j = 1, \dots, 17$ (1.138 for $j = 1$): the starting factor of the chain-ladder start, and the factor of every network that rule (12) leaves at its start, has a variance about 12% larger than Mack’s factor on all learning cells would have. A network that does move inherits the same handicap in its level.

There is a second effect that (20) does not show. If the latest accident years develop differently, the factor without them is off in the direction that matters for the reserve, because the cells to be developed in (16) are the recent accident years. The table of Section 4 shows the size: for $j = 1$ the training factor is 2.535, the factor of all learning cells 2.528.

The one-stage network is the one that was validated. Guarantee (13) is a statement about $\hat{\theta}_j$ itself. In the two-stage design the validated network $\theta_{t^*}(\mathcal{T}, s)$ is thrown away, and the network that is used, $\theta_{t^*}(\mathcal{T} \cup \mathcal{V}, s)$, has no validation loss: all data went into it. One relies on the number of steps being transferable from the smaller to the larger data set. With full-batch steps this is plausible, because the gradient is a mean over the rows and has the same scale on both sets, but it is an assumption and not a guarantee.

	one stage (18)	two stages (19)
rows in the gradient of the final network	$\mathcal{T}_j$ (90%)	all of $\mathcal{L}_j$
validation loss of the final network	known, and $\leq$ the start’s by (13)	does not exist
latest accident years	monitor only	estimate the factors
fits per network	one	two

10.3 What a refit would look like here

The code does not do this; it is written out because it is the direct analogue of (19). For network j , with t^* from the one-stage run:

1. start at the factor of all learning cells, $\beta_0 = \log \hat{f}_{j-1}^{\mathcal{L}}$, $\beta = \mathbf{0}$, same seed;
2. run t^* steps (7) with the gradient of the loss on all of $\mathcal{L}_j$ in place of $\ell^{\mathcal{T}}$;
3. use the resulting factors in (16).

For the networks with $t^* = 0$ this replaces $\hat{f}_{j-1}^{\mathcal{T}}$ by $\hat{f}_{j-1}^{\mathcal{L}}$ and nothing else.

11 Properties of this particular split

Four consequences of cutting the rows at a fixed position, each a direct reading of Section 4.

1. **Small validation sets in the late periods.** $n_j^{\mathcal{V}}$ falls from 290 to 16, and from $j = 10$ on the validation rows come from a single accident year. For $j = 18$ and 19 they carry 3.4% and 0.3% of the volume. The stopping time t^* of those networks rests on very little.
2. **The validation rows are not exchangeable with the training rows.** They differ in accident year (later) and, when a year is cut, in the covariates (the end of the order $\prec$). A validation loss that falls therefore says that the network improves on *those* cells; whether that carries over to the diagonal cells in (16) depends on how similar they are.
3. **A cell changes its role with j .** A cell of accident year 10 is a training row of the networks 1, ..., 9 (it is far from the end of their learning cells) and can be a validation row of network 10, where its accident year is the last. No cell is “a validation cell” as such.
4. **The benchmark uses a different set.** The homogeneous model in the tables (`f_hom`, and its loss) is $\hat{f}_{j-1}^{\mathcal{L}}$ on all learning cells, whereas the chain-ladder start is $\hat{f}_{j-1}^{\mathcal{T}}$. By (5) they differ by $\omega_j(\hat{f}_{j-1}^{\mathcal{V}} - \hat{f}_{j-1}^{\mathcal{T}})$. A chain-ladder-start network that never moves therefore does not reproduce the homogeneous loss exactly.

The seed does not enter the split. It changes θ_0 (the random hidden weights), and through the path (7) it changes t^* and $\hat{\theta}_j$; the sets $\mathcal{T}_j$ and $\mathcal{V}_j$ stay the same. The spread over the seeds in the tables is therefore the variability of the optimisation, not of the split.

12 Where each step is in the code

Step	Formula	Code
observed and future payments	(1)	fit script: <code>observed</code> ; analysis script: <code>cells\$true</code>
learning cells	(3)	fit script: <code>learn_rows</code>
order of the rows	$\prec$	fit script: <code>keyby = c("i", features)</code>
training and validation rows	(4)	Keras <code>validation_split</code> ; <code>nncl_fit()</code> : <code>train</code> , <code>vali</code>
losses	(6)	Keras <code>loss = "mse"</code> , <code>val_loss</code> in history
one step per epoch	(7)	<code>batch_size 10,000</code> in <code>config.yml</code>
paper's training	(8)	<code>param (early_stop = FALSE, epochs 100)</code>

Step	Formula	Code
early stopping	(9)-(10)	<code>callback_early_stopping(patience, restore_best_weights)</code>
chain-ladder start	(11)	fit script: <code>f_start</code> ; <code>nncl_model(): set_weights</code>
keeping the start	(12)	<code>nncl_fit(): mu_start, set_weights(model, start)</code>
reported losses	(15)	<code>nncl_fit(): loss, loss_train, loss_vali</code>
diagonal cells, reserve	(16)	fit script: <code>diag_rows, x_diag</code> ; analysis script: <code>f_prod, nncl_reserves()</code>
truth and bias	(17)	analysis script: <code>true_total, bias_pct</code>

13 Numerical check

All checks are in base R on the saved cells and fits; “CL start” is the one-layer network of 20 neurons with Adam and the training `cl_start`, 20 seeds times 19 development periods = 380 fits; “paper” is the one-layer network with RMSprop and the training `paper`, also 380 fits.

Statement	Result
sizes of $\mathcal{O}$ and $\mathcal{O}^c$; diagonal cells with a positive and a zero payment	34,932 and 31,908; 3,167 and 16
table of Section 4 (n_j, n_j^T, n_j^V , accident years, volume shares, factors)	recomputed from the cells with the cut (4)
(15): <code>loss = loss_train + loss_vali</code> , CL start	relative difference at most $2 \cdot 10^{-16}$ in all 380 fits
(15): <code>loss_vali</code> recomputed from the saved factors on the rows of $\mathcal{V}_j$	relative difference at most $2 \cdot 10^{-11}$ in all 380 fits
(9)-(10): $t_{\text{stop}} = t^* + 20$ when training stops before 500 epochs with $t^* \geq 1$, CL start	228 of 228 fits
(10): the saved network is the one of epoch t^* , also when all 500 epochs were run (all <code>cl_start</code> combinations)	33 such fits; saved validation loss equals that of epoch t^* to 10^{-7} and differs from that of epoch 500
(12): fits kept at the start, CL start	152 of 380; always for $j = 15, 17, 18, 19$, in 95% of the seeds for $j = 1$ and 16, never for $j = 2, \dots, 8$
(13): validation loss of the trained fits ($t^* \geq 1$) below that of the start	228 of 228; on average 3.3% below, at most 23.9%
(13): validation loss of the fits kept at the start	equal to the start’s up to single-precision rounding
(8): paper’s training, epochs run and used	100 and 100 in all 380 fits
paper’s training: best validation epoch earlier than epoch 100	207 of 380 fits (the validation loss is not used)
paper’s training: final validation loss above that of the chain-ladder factor $\hat{f}_{j-1}^T$	328 of 380 fits
(20): variance ratio $1/(1 - \omega_j)$	1.09 to 1.14 for $j = 1, \dots, 17$; 1.04 and 1.00 for $j = 18, 19$

The statements about Keras’s behaviour (the cut without shuffling, the shuffling of the training rows afterwards, the strict improvement required by the early-stopping callback) are from the Keras

documentation and are consistent with every check above; the bCCNN procedure of Section 10 is as in your script `analysis/03_bCCNN/bCCNN_fit.R` with `final_fit: refit`.